

COMPLETE_API_REFERENCE

HelixCode Complete API Reference

Overview

HelixCode provides a comprehensive REST API for enterprise AI development platform operations. This reference documents all available endpoints, authentication methods, request/response formats, and usage examples.

Table of Contents

- [Authentication](#)
- [Projects](#)
- [Tasks](#)
- [Sessions](#)
- [Users](#)
- [LLM Providers](#)
- [Workers](#)
- [Files](#)
- [Memory](#)
- [Notifications](#)
- [Configuration](#)
- [System](#)
- [Monitoring](#)
- [Security](#)

Authentication

POST /api/v1/auth/login

Authenticate a user and receive a JWT token.

Request Body:

```
{
  "username": "string",
  "password": "string",
  "mfa_code": "string (optional)"
}
```

Response:

```
{
  "success": true,
  "token": "jwt_token_string",
  "expires_at": "2024-12-31T23:59:59Z",
  "user": {
```

```
    "id": "uuid",
    "username": "string",
    "email": "string",
    "role": "string"
  }
}
```

Status Codes: - 200 - Success - 401 - Invalid credentials - 429 - Too many attempts

POST /api/v1/auth/register

Register a new user account.

Request Body:

```
{
  "username": "string",
  "email": "string",
  "password": "string",
  "display_name": "string (optional)"
}
```

Response:

```
{
  "success": true,
  "user": {
    "id": "uuid",
    "username": "string",
    "email": "string",
    "verification_required": true
  }
}
```

POST /api/v1/auth/logout

Invalidate the current session token.

Headers:

Authorization: Bearer <token>

Response:

```
{
  "success": true,
  "message": "Logged out successfully"
}
```

POST /api/v1/auth/refresh

Refresh an expired JWT token.

Request Body:

```
{
  "refresh_token": "string"
}
```

Response:

```
{
  "success": true,
  "token": "new_jwt_token",
  "expires_at": "2024-12-31T23:59:59Z"
}
```

Projects

GET /api/v1/projects

List all projects for the authenticated user.

Query Parameters: - `limit` (integer, optional): Maximum number of projects to return (default: 50) - `offset` (integer, optional): Number of projects to skip (default: 0) - `status` (string, optional): Filter by status (active, archived, deleted)

Response:

```
{
  "projects": [
    {
      "id": "uuid",
      "name": "string",
      "description": "string",
      "status": "active",
      "created_at": "2024-01-01T00:00:00Z",
      "updated_at": "2024-01-01T00:00:00Z",
      "owner_id": "uuid"
    }
  ],
  "total": 25,
  "limit": 50,
  "offset": 0
}
```

POST /api/v1/projects

Create a new project.

Request Body:

```
{
  "name": "string",
  "description": "string (optional)",
  "template": "string (optional)",
  "settings": {
    "visibility": "private|public|team",
    "version_control": "git",
    "ci_cd_enabled": true
  }
}
```

Response:

```
{
  "success": true,
  "project": {
    "id": "uuid",
    "name": "string",
    "description": "string",
    "status": "active",
    "created_at": "2024-01-01T00:00:00Z",
    "owner_id": "uuid"
  }
}
```

GET /api/v1/projects/{project_id}

Get detailed information about a specific project.

Path Parameters: - project_id (uuid): Project identifier

Response:

```
{
  "project": {
    "id": "uuid",
    "name": "string",
    "description": "string",
    "status": "active",
    "created_at": "2024-01-01T00:00:00Z",
    "updated_at": "2024-01-01T00:00:00Z",
    "owner_id": "uuid",
    "collaborators": [
      {
        "user_id": "uuid",
        "username": "string",
        "role": "owner|editor|viewer",
        "added_at": "2024-01-01T00:00:00Z"
      }
    ]
  }
}
```

```

    }
  ],
  "statistics": {
    "total_tasks": 150,
    "completed_tasks": 120,
    "active_sessions": 3,
    "storage_used_mb": 250
  }
}

```

PUT /api/v1/projects/{project_id}

Update project information.

Request Body:

```

{
  "name": "string (optional)",
  "description": "string (optional)",
  "status": "active|archived|deleted (optional)"
}

```

DELETE /api/v1/projects/{project_id}

Delete a project.

Status Codes: - 204 - Successfully deleted - 403 - Not authorized - 404 - Project not found

Tasks

GET /api/v1/tasks

List tasks with optional filtering.

Query Parameters: - `project_id` (uuid, optional): Filter by project - `status` (string, optional): Filter by status (pending, running, completed, failed) - `priority` (string, optional): Filter by priority (low, normal, high, critical) - `limit` (integer, optional): Maximum results (default: 50) - `offset` (integer, optional): Pagination offset (default: 0)

Response:

```

{
  "tasks": [
    {
      "id": "uuid",
      "project_id": "uuid",
      "type": "code_generation",
      "status": "completed",
      "priority": "normal",

```

```
    "title": "Generate user authentication module",
    "description": "Create login, registration, and session
        management",
    "created_at": "2024-01-01T00:00:00Z",
    "updated_at": "2024-01-01T00:00:00Z",
    "assigned_worker": "uuid",
    "progress": 100,
    "estimated_duration": 300
  }
],
"total": 150,
"limit": 50,
"offset": 0
}
```

POST /api/v1/tasks

Create a new task.

Request Body:

```
{
  "project_id": "uuid",
  "type": "code_generation|code_review|testing|deployment",
  "title": "string",
  "description": "string (optional)",
  "priority": "low|normal|high|critical",
  "parameters": {
    "language": "go",
    "framework": "gin",
    "database": "postgresql",
    "additional_requirements": ["authentication", "logging"]
  },
  "timeout": 3600,
  "dependencies": ["uuid1", "uuid2"]
}
```

Response:

```
{
  "success": true,
  "task": {
    "id": "uuid",
    "project_id": "uuid",
    "type": "code_generation",
    "status": "pending",
    "priority": "normal",
    "title": "string",
    "created_at": "2024-01-01T00:00:00Z",
```

```
    "estimated_duration": 1800
  }
}
```

GET /api/v1/tasks/{task_id}

Get detailed task information.

Response:

```
{
  "task": {
    "id": "uuid",
    "project_id": "uuid",
    "type": "code_generation",
    "status": "running",
    "priority": "normal",
    "title": "Generate REST API",
    "description": "Create CRUD endpoints for user management",
    "created_at": "2024-01-01T00:00:00Z",
    "updated_at": "2024-01-01T00:00:00Z",
    "started_at": "2024-01-01T00:05:00Z",
    "completed_at": null,
    "assigned_worker": "uuid",
    "progress": 65,
    "estimated_duration": 1800,
    "actual_duration": 1170,
    "parameters": {
      "language": "go",
      "endpoints": ["GET /users", "POST /users", "PUT /users/{id}", "DELETE /users/{id}"],
      "authentication": true,
      "validation": true
    },
  },
  "result": {
    "files_generated": 3,
    "lines_of_code": 245,
    "test_coverage": 85
  },
  "logs": [
    {
      "timestamp": "2024-01-01T00:05:00Z",
      "level": "info",
      "message": "Starting code generation for REST API"
    }
  ]
}
```

GET /api/v1/tasks/{task_id}/status

Get current task execution status.

Response:

```
{
  "task_id": "uuid",
  "status": "running",
  "progress": 65,
  "stage": "Generating API handlers",
  "estimated_completion": "2024-01-01T00:25:00Z",
  "worker_info": {
    "worker_id": "uuid",
    "hostname": "worker-01.helixcode.internal",
    "load_average": 0.75
  }
}
```

POST /api/v1/tasks/{task_id}/cancel

Cancel a running task.

Response:

```
{
  "success": true,
  "message": "Task cancellation requested",
  "task_id": "uuid"
}
```

Sessions

GET /api/v1/sessions

List active development sessions.

Query Parameters: - `project_id` (uuid, optional): Filter by project - `user_id` (uuid, optional): Filter by user - `status` (string, optional): Filter by status (active, paused, completed)

Response:

```
{
  "sessions": [
    {
      "id": "uuid",
      "project_id": "uuid",
      "user_id": "uuid",
      "type": "development",
      "status": "active",
    }
  ]
}
```

```

    "created_at": "2024-01-01T00:00:00Z",
    "last_activity": "2024-01-01T00:15:00Z",
    "context": {
      "current_file": "/src/main.go",
      "cursor_position": {"line": 25, "column": 10},
      "open_files": ["/src/main.go", "/src/auth.go", "/src/database.go"]
    }
  },
  "total": 5
}

```

POST /api/v1/sessions

Create a new development session.

Request Body:

```

{
  "project_id": "uuid",
  "type": "development|debugging|review",
  "initial_context": {
    "files": ["/src/main.go"],
    "cursor_position": {"line": 1, "column": 1}
  },
  "settings": {
    "auto_save": true,
    "real_time_collaboration": true,
    "llm_assistance": true
  }
}

```

GET /api/v1/sessions/{session_id}

Get session details and context.

WebSocket: /ws/sessions/{session_id}

Real-time session collaboration WebSocket endpoint.

Message Types: - `cursor_update`: Cursor position changes - `file_change`: File content modifications - `user_join`: User joining session - `user_leave`: User leaving session - `llm_suggestion`: AI-generated code suggestions

LLM Providers

GET /api/v1/providers

List available LLM providers and their status.

Response:

```
{
  "providers": [
    {
      "id": "openai",
      "name": "OpenAI",
      "type": "cloud",
      "models": ["gpt-4", "gpt-3.5-turbo", "gpt-4-turbo"],
      "status": "available",
      "latency_ms": 1200,
      "cost_per_token": 0.00015,
      "rate_limit": {
        "requests_per_minute": 60,
        "tokens_per_minute": 40000
      }
    },
    {
      "id": "anthropic",
      "name": "Anthropic Claude",
      "type": "cloud",
      "models": ["claude-3-opus", "claude-3-sonnet", "claude-3-haiku"],
      "status": "available",
      "latency_ms": 1800,
      "cost_per_token": 0.00025,
      "rate_limit": {
        "requests_per_minute": 50,
        "tokens_per_minute": 30000
      }
    }
  ]
}
```

POST /api/v1/providers/{provider_id}/generate

Generate text using a specific LLM provider.

Request Body:

```
{
  "model": "gpt-4",
  "prompt": "Write a Go function to validate email addresses",
  "max_tokens": 500,
  "temperature": 0.7,
  "top_p": 1.0,
  "frequency_penalty": 0.0,
  "presence_penalty": 0.0,
  "stop_sequences": ["\\n\\n", "###"],
}
```

```
    "stream": false
}
```

Response:

```
{
  "success": true,
  "provider": "openai",
  "model": "gpt-4",
  "generated_text": "func ValidateEmail(email string) bool {\n    \n    // Email validation logic\n    return true\n}",
  "usage": {
    "prompt_tokens": 15,
    "completion_tokens": 25,
    "total_tokens": 40
  },
  "latency_ms": 1250,
  "cost_usd": 0.006
}
```

WebSocket: /ws/providers/{provider_id}/stream

Streaming text generation endpoint.

Request:

```
{
  "model": "gpt-4",
  "prompt": "Write a comprehensive README for a Go project",
  "max_tokens": 2000,
  "stream": true
}
```

Streaming Response:

```
{
  "type": "chunk",
  "content": "# Project Title\n\nThis is a Go project...",
  "finished": false
}
{
  "type": "usage",
  "usage": {
    "prompt_tokens": 20,
    "completion_tokens": 450,
    "total_tokens": 470
  }
}
{
  "type": "done",
```

```
    "finished": true
}
```

Workers

GET /api/v1/workers

List all registered workers and their status.

Response:

```
{
  "workers": [
    {
      "id": "uuid",
      "hostname": "worker-01.helixcode.internal",
      "ip_address": "10.0.1.100",
      "platform": "linux",
      "architecture": "amd64",
      "status": "active",
      "capabilities": ["go", "python", "docker", "gpu"],
      "current_load": {
        "cpu_percent": 45.2,
        "memory_percent": 62.1,
        "active_tasks": 3
      },
      "last_heartbeat": "2024-01-01T00:00:30Z",
      "registered_at": "2024-01-01T00:00:00Z"
    }
  ],
  "total": 5,
  "active": 4,
  "idle": 1
}
```

POST /api/v1/workers/register

Register a new worker node.

Request Body:

```
{
  "hostname": "worker-01.company.com",
  "ip_address": "10.0.1.100",
  "platform": "linux",
  "architecture": "amd64",
  "capabilities": ["go", "python", "docker", "kubernetes"],
  "resources": {
    "cpu_cores": 8,
```

```
    "memory_gb": 16,  
    "storage_gb": 100,  
    "gpu_count": 1  
  },  
  "tags": ["gpu-enabled", "high-memory"]  
}
```

GET /api/v1/workers/{worker_id}

Get detailed worker information and metrics.

POST /api/v1/workers/{worker_id}/drain

Put worker in drain mode (finish current tasks but don't accept new ones).

DELETE /api/v1/workers/{worker_id}

Unregister a worker.

Files

GET /api/v1/files

List files in a project or directory.

Query Parameters: - `project_id` (uuid): Project identifier - `path` (string, optional): Directory path (default: `"/`) - `recursive` (boolean, optional): Include subdirectories (default: `false`)

Response:

```
{  
  "files": [  
    {  
      "name": "main.go",  
      "path": "/src/main.go",  
      "type": "file",  
      "size": 2048,  
      "modified_at": "2024-01-01T00:00:00Z",  
      "permissions": "0644",  
      "language": "go"  
    },  
    {  
      "name": "src",  
      "path": "/src",  
      "type": "directory",  
      "size": 0,  
      "modified_at": "2024-01-01T00:00:00Z",  
      "permissions": "0755"  
    }  
  ]  
}
```

```
],
"total": 25,
"path": "/src"
}
```

GET /api/v1/files/content

Get file content.

Query Parameters: - `project_id` (uuid): Project identifier - `path` (string): File path

Response:

```
{
  "path": "/src/main.go",
  "content": "package main\\n\\nimport (\\n    \\\"fmt\\\"\\n)\\n\\n\\nfunc main() {\\n    fmt.Println(\\\"Hello, World!\\\")\\n}\\n\\n",
  "encoding": "utf-8",
  "size": 2048,
  "modified_at": "2024-01-01T00:00:00Z",
  "language": "go",
  "line_count": 8
}
```

PUT /api/v1/files/content

Update file content.

Request Body:

```
{
  "project_id": "uuid",
  "path": "/src/main.go",
  "content": "package main\\n\\nimport (\\n    \\\"fmt\\\"\\n    \\\"log\\\"\\n)\\n\\nfunc main() {\\n    log.Println(\\\"Starting application\\\")\\n    fmt.Println(\\\"Hello, World!\\\")\\n}\\n",
  "encoding": "utf-8"
}
```

POST /api/v1/files/upload

Upload a file or create a new file.

Content-Type: multipart/form-data

Form Fields: - project_id: Project UUID - path: Target file path - file: File content (binary) - overwrite (optional): Overwrite existing file (default: false)

DELETE /api/v1/files

Delete a file or directory.

Query Parameters: - `project_id` (uuid): Project identifier - `path` (string): File/directory path - `recursive` (boolean, optional): Delete directories recursively (default: false)

Memory

GET /api/v1/memory/providers

List available memory providers.

Response:

```
{
  "providers": [
    {
      "name": "chromadb",
      "type": "vector_database",
      "status": "connected",
      "collections": 5,
      "total_vectors": 12500,
      "capabilities": ["semantic_search", "metadata_filtering",
        "batch_operations"]
    }
  ]
}
```

POST /api/v1/memory/store

Store data in memory.

Request Body:

```
{
  "provider": "chromadb",
  "collection": "code_snippets",
  "data": {
    "content": "func ValidateEmail(email string) bool { return true }",
    "language": "go",
    "type": "function",
    "tags": ["validation", "email"]
  },
  "metadata": {
    "author": "user123",
    "created_at": "2024-01-01T00:00:00Z",
    "project": "auth-service"
  }
}
```

```
}  
}
```

POST /api/v1/memory/search

Search memory content.

Request Body:

```
{  
  "provider": "chromadb",  
  "collection": "code_snippets",  
  "query": "email validation function",  
  "limit": 10,  
  "filters": {  
    "language": "go",  
    "type": "function"  
  },  
  "include_metadata": true  
}
```

Response:

```
{  
  "results": [  
    {  
      "id": "uuid",  
      "content": "func ValidateEmail(email string) bool { return  
        true }",  
      "score": 0.95,  
      "metadata": {  
        "language": "go",  
        "type": "function",  
        "author": "user123"  
      }  
    }  
  ],  
  "total": 1,  
  "query_time_ms": 45  
}
```

Notifications

GET /api/v1/notifications

Get user notifications.

Query Parameters: - `status` (string, optional): Filter by status (unread, read, archived) - `type` (string, optional): Filter by type (task_completed, error, warning, info) - `limit` (integer, optional): Maximum results (default: 50)

Response:

```
{
  "notifications": [
    {
      "id": "uuid",
      "type": "task_completed",
      "title": "Code Generation Complete",
      "message": "Your authentication module has been generated successfully",
      "status": "unread",
      "created_at": "2024-01-01T00:00:00Z",
      "data": {
        "task_id": "uuid",
        "project_id": "uuid",
        "files_generated": 3
      }
    }
  ],
  "total": 25,
  "unread": 5
}
```

POST /api/v1/notifications/{notification_id}/read

Mark notification as read.

PUT /api/v1/notifications/settings

Update notification preferences.

Request Body:

```
{
  "email_notifications": true,
  "slack_notifications": true,
  "webhook_url": "https://hooks.slack.com/...",
  "notification_types": {
    "task_completed": true,
    "task_failed": true,
    "system_alerts": true,
    "security_events": true
  }
}
```

Configuration

GET /api/v1/config

Get current system configuration.

Response:

```
{
  "server": {
    "host": "0.0.0.0",
    "port": 8080,
    "tls_enabled": true,
    "rate_limit": 1000
  },
  "database": {
    "type": "postgresql",
    "host": "localhost",
    "port": 5432,
    "database": "helixcode"
  },
  "llm": {
    "default_provider": "openai",
    "max_tokens": 4096,
    "temperature": 0.7
  },
  "workers": {
    "max_concurrent_tasks": 10,
    "health_check_interval": 30
  }
}
```

PUT /api/v1/config

Update system configuration.

Request Body:

```
{
  "llm": {
    "default_provider": "anthropic",
    "temperature": 0.8
  },
  "workers": {
    "max_concurrent_tasks": 15
  }
}
```

POST /api/v1/config/reload

Reload configuration from files.

System

GET /api/v1/health

Get system health status.

Response:

```
{
  "status": "healthy",
  "timestamp": "2024-01-01T00:00:00Z",
  "version": "1.0.0",
  "uptime": "24h30m45s",
  "components": {
    "database": {
      "status": "healthy",
      "latency_ms": 5,
      "connections": 8
    },
    "redis": {
      "status": "healthy",
      "latency_ms": 2,
      "memory_usage": "45%"
    },
    "workers": {
      "status": "healthy",
      "active": 4,
      "total": 5
    }
  }
}
```

GET /api/v1/system/info

Get detailed system information.

Response:

```
{
  "system": {
    "hostname": "helixcode-server-01",
    "platform": "linux",
    "architecture": "amd64",
    "cpu_cores": 8,
    "memory_total_gb": 16,
  }
}
```

```
    "disk_total_gb": 100,
    "load_average": [1.25, 1.15, 1.05]
  },
  "application": {
    "version": "1.0.0",
    "build_date": "2024-01-01T00:00:00Z",
    "go_version": "go1.26.3",
    "git_commit": "abc123def456"
  },
  "runtime": {
    "goroutines": 125,
    "memory_allocated_mb": 256,
    "memory_used_mb": 180,
    "gc_cycles": 45,
    "gc_pause_ms": 12.5
  }
}
```

GET /api/v1/system/metrics

Get detailed system metrics.

Response:

```
{
  "cpu": {
    "usage_percent": 45.2,
    "cores": 8,
    "load_average": [1.25, 1.15, 1.05]
  },
  "memory": {
    "total_gb": 16,
    "used_gb": 8.5,
    "available_gb": 7.5,
    "usage_percent": 53.1
  },
  "disk": {
    "total_gb": 100,
    "used_gb": 45.2,
    "available_gb": 54.8,
    "usage_percent": 45.2
  },
  "network": {
    "bytes_sent_mb": 1250,
    "bytes_received_mb": 980,
    "connections_active": 45,
    "connections_total": 1250
  }
}
```

Monitoring

GET /api/v1/monitoring/metrics

Get application performance metrics.

Response:

```
{
  "requests": {
    "total": 15420,
    "per_second": 12.5,
    "avg_response_time_ms": 245,
    "p95_response_time_ms": 450,
    "p99_response_time_ms": 890,
    "error_rate_percent": 0.8
  },
  "tasks": {
    "total_created": 1250,
    "total_completed": 1180,
    "avg_completion_time_s": 180,
    "success_rate_percent": 94.4,
    "queue_length": 5
  },
  "llm": {
    "total_requests": 2500,
    "avg_latency_ms": 1250,
    "tokens_used": 125000,
    "cost_usd": 15.75
  },
  "workers": {
    "total_workers": 5,
    "active_workers": 4,
    "avg_load_percent": 65.2,
    "tasks_distributed": 1180
  }
}
```

GET /api/v1/monitoring/logs

Get application logs.

Query Parameters: - `level` (string, optional): Log level filter (debug, info, warn, error) - `service` (string, optional): Service filter - `start_time` (string, optional): Start time (RFC3339) - `end_time` (string, optional): End time (RFC3339) - `limit` (integer, optional): Maximum results (default: 100)

Response:

```
{
  "logs": [
    {
      "timestamp": "2024-01-01T00:00:00Z",
      "level": "info",
      "service": "api",
      "message": "Task completed successfully",
      "fields": {
        "task_id": "uuid",
        "duration_ms": 2500,
        "user_id": "uuid"
      }
    }
  ],
  "total": 150,
  "has_more": true
}
```

POST /api/v1/monitoring/alerts

Create a monitoring alert.

Request Body:

```
{
  "name": "High Error Rate",
  "condition": "error_rate > 5%",
  "severity": "critical",
  "channels": ["slack", "email"],
  "cooldown_minutes": 15,
  "enabled": true
}
```

Security

GET /api/v1/security/audit

Get security audit logs.

Query Parameters: - `user_id` (uuid, optional): Filter by user - `action` (string, optional): Filter by action - `resource` (string, optional): Filter by resource - `start_time` (string, optional): Start time filter - `end_time` (string, optional): End time filter

Response:

```
{
  "audit_logs": [
    {
      "id": "uuid",
      "timestamp": "2024-01-01T00:00:00Z",
```

```
    "user_id": "uuid",
    "username": "john_doe",
    "action": "login",
    "resource": "auth",
    "ip_address": "192.168.1.100",
    "user_agent": "Mozilla/5.0...",
    "success": true,
    "details": {
      "method": "password",
      "mfa_used": true
    }
  },
  "total": 500
}
```

POST /api/v1/security/scan

Initiate security scan.

Request Body:

```
{
  "scan_type": "full_system",
  "targets": ["api", "database", "workers"],
  "severity_threshold": "medium",
  "include_compliance_checks": true,
  "generate_report": true
}
```

Response:

```
{
  "scan_id": "uuid",
  "status": "running",
  "estimated_completion": "2024-01-01T00:30:00Z",
  "targets": ["api", "database", "workers"]
}
```

GET /api/v1/security/scan/{scan_id}/results

Get security scan results.

Response:

```
{
  "scan_id": "uuid",
  "status": "completed",
  "completed_at": "2024-01-01T00:25:00Z",
  "summary": {
```

```

    "total_issues": 3,
    "critical": 0,
    "high": 1,
    "medium": 2,
    "low": 0
  },
  "issues": [
    {
      "id": "uuid",
      "severity": "high",
      "title": "Weak password policy",
      "description": "Password requirements are too lenient",
      "category": "authentication",
      "recommendation": "Require minimum 12 characters with
        special symbols",
      "cve": null,
      "affected_components": ["auth_service"]
    }
  ],
  "compliance": {
    "gdpr_compliant": true,
    "hipaa_compliant": true,
    "pci_compliant": false,
    "iso27001_compliant": true
  }
}

```

Error Responses

All API endpoints follow consistent error response format:

```

{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request parameters",
    "details": {
      "field": "username",
      "reason": "required field missing"
    },
    "request_id": "req_abc123def456",
    "timestamp": "2024-01-01T00:00:00Z"
  }
}

```

Common Error Codes

- `VALIDATION_ERROR` (400): Invalid request parameters
- `AUTHENTICATION_ERROR` (401): Authentication required or failed

- `AUTHORIZATION_ERROR` (403): Insufficient permissions
- `NOT_FOUND` (404): Resource not found
- `CONFLICT` (409): Resource conflict
- `RATE_LIMITED` (429): Too many requests
- `INTERNAL_ERROR` (500): Server error
- `SERVICE_UNAVAILABLE` (503): Service temporarily unavailable

Rate Limiting

API endpoints are rate limited based on user tier:

- **Free Tier:** 100 requests/hour, 1000 requests/day
- **Pro Tier:** 1000 requests/hour, 10000 requests/day
- **Enterprise Tier:** 10000 requests/hour, unlimited daily

Rate limit headers are included in all responses:

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 950
X-RateLimit-Reset: 1640995200
X-RateLimit-Retry-After: 60 (only when limited)
```

Pagination

List endpoints support pagination:

Query Parameters: - `limit`: Maximum items per page (default: 50, max: 1000) - `offset`: Number of items to skip (default: 0) - `cursor`: Cursor for efficient pagination (preferred over offset)

Response Format:

```
{
  "data": [...],
  "pagination": {
    "total": 1250,
    "limit": 50,
    "offset": 100,
    "has_more": true,
    "next_cursor": "cursor_abc123"
  }
}
```

Webhooks

Configure webhooks for real-time notifications:

POST /api/v1/webhooks

Register a webhook endpoint.

Request Body:

```
{
  "url": "https://your-app.com/webhooks/helixcode",
  "secret": "your_webhook_secret",
  "events": ["task.completed", "task.failed", "project.created"],
  "active": true
}
```

Webhook Payload Format

```
{
  "event": "task.completed",
  "timestamp": "2024-01-01T00:00:00Z",
  "data": {
    "task_id": "uuid",
    "project_id": "uuid",
    "user_id": "uuid",
    "status": "completed",
    "result": {...}
  },
  "signature": "sha256=abc123..."
}
```

SDKs and Libraries

Official SDKs are available for popular languages:

- **Go:** `go get dev.helixcode/sdk/go`
- **Python:** `pip install helixcode-sdk`
- **JavaScript/TypeScript:** `npm install @helixcode/sdk`
- **Java:** Maven/Gradle dependencies available

Versioning

API versioning follows semantic versioning:

- **v1:** Current stable version
- **v2:** Next major version (breaking changes)
- **Beta endpoints:** Prefixed with `/beta/`

Specify version in request headers:

Accept: `application/vnd.helixcode.v1+json`

Support

For API support and questions:

- **Documentation:** <https://docs.helixcode.ai>

- **API Status:** <https://status.helixcode.ai>
 - **Developer Forum:** <https://community.helixcode.ai>
 - **Email Support:** api-support@helixcode.ai
-

Last updated: December 2024 docs/COMPLETE_API_REFERENCE.md ## Sources verified
Sources verified 2026-05-29: <https://go.dev/doc/devel/release> (Go 1.26.3 confirmed latest stable;
the illustrative `go_version` field in the health-endpoint JSON example corrected `go1.21.5` →
`go1.26.3` to match inner `go.mod` `go 1.26` / CLAUDE.md §3.1) — this is primarily an internal
HelixCode HTTP API reference (endpoints, request/response shapes) rather than third-party-
service instructions; the only external version claim was the Go build-info example, now current.
Project version authority is `go.mod` (inner module `go 1.26`) + CLAUDE.md §3.1.